

Software Project Management

Tuesday, October 14, 2025 2:51 PM

Software Project Management

Software project management is the process of planning, leading, and controlling software projects. It is both an art and a science because it involves managing people, tasks, and technical challenges. While good management cannot guarantee success, poor management often leads to project failure.

Why software project management is different from other projects:

- Software is intangible; it cannot be physically seen or touched.
- There are no universal, standardized software processes; methods vary by project.
- Large software projects are often unique, one-time efforts.

Example: Developing a mobile banking app requires careful planning, assigning tasks, monitoring progress, and addressing unexpected challenges. Without good management, the project may miss deadlines or have serious errors

Software Project Manager

A software project manager is the person responsible for the overall planning, execution, and control of a software project. They schedule tasks, allocate resources, manage risks, and ensure the project meets its goals.

Example: A project manager assigns a developer to create the login module of an app while another works on the transaction system, ensuring both parts are integrated on time.

Project Management Activities

1. Project Planning

Project planning is the process of preparing everything needed for a project to succeed. It involves dividing the work into smaller tasks, assigning responsibilities, organizing resources, and anticipating potential problems.

Sections of Project Planning:

- **Introduction:** Sets the project objectives and constraints like time and budget.
- **Project Organization:** Lists team members and their roles.
- **Risk Analysis:** Identifies possible problems, like delays or technical issues.
- **Hardware and Software Requirements:** Specifies the tools, platforms, and equipment needed.
- **Work Breakdown:** Breaks the project into activities, milestones, and deliverables.
- **Project Schedule:** Shows which tasks depend on others and the sequence of work.
- **Monitoring and Reporting:** Defines how progress will be tracked and reported.

Example: Before starting a banking app, the team decides the objectives (secure, fast, user-friendly), lists tasks (login, transactions, notifications), assigns developers, estimates completion times, identifies risks like server downtime, and schedules testing milestones.

Types of Project Plans

- **Quality Plan:** Defines quality standards and procedures to ensure the product works correctly.
Example: Testing the banking app to ensure there are no login errors.
- **Validation Plan:** Defines how the system will be tested to meet requirements.
Example: Checking that the app handles all types of transactions correctly.
- **Configuration Management Plan:** Manages changes in the project, including version control of code.
Example: Using Git to track updates to the app's source code.
- **Maintenance Plan:** Predicts future maintenance needs, costs, and effort.
Example: Planning for regular updates and bug fixes after the app is launched.

- **Staff Development Plan:** Plans how to improve team skills and knowledge during the project.
Example: Training developers in new security protocols for online banking.

2. Project Scheduling

Project scheduling is the process of breaking down the total work of a project into smaller activities and estimating the time required for each. It creates a detailed timeline showing when each activity should start and finish, and how much resource (people, tools, money) will be needed at each stage.

Purpose:

The main goal of scheduling is to ensure that all project tasks are completed in the correct order, without delays or resource conflicts.

Example:

In a software company developing an online learning platform, the schedule might show:

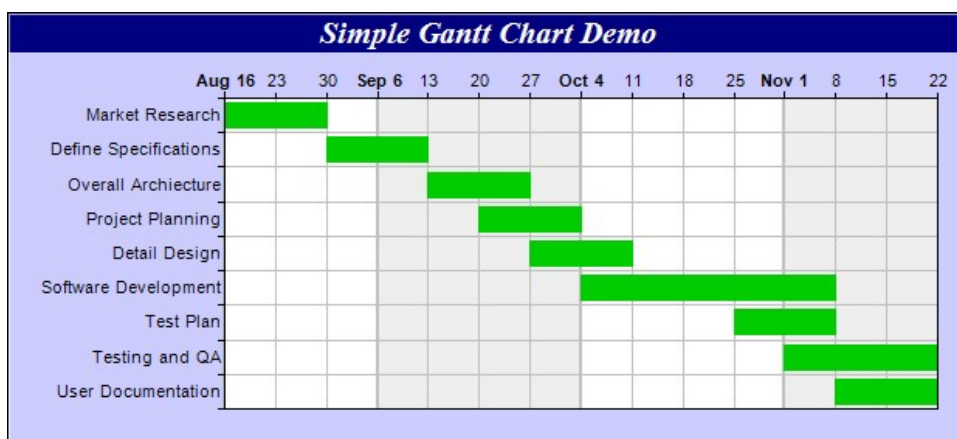
- UI design: Week 1–2
- Backend coding: Week 3–6
- Testing: Week 7–8
- Deployment: Week 9

Stages in Creating a Project Schedule:

- Constructing an Initial Activity Plan:**
 - Identify all the activities required for the project.
 - Define their order and dependencies (which task must be completed before another starts).
- Risk Analysis:**
 - Analyze possible risks that could delay tasks, such as resource shortage or technical issues.
- Resource Allocation:**
 - Assign resources like developers, testers, tools, and hardware to specific activities.
- Schedule Production:**
 - Create the final version of the schedule, showing start/end dates and responsibilities.

Representation of Project Schedule:

Chart Type	Description	Example Use
Bar Chart (Gantt Chart)	Shows activities, their duration, and who is responsible.	Useful to visualize overlapping tasks (e.g., coding and design running together).
Activity Network (PERT/CPM Chart)	Shows dependencies between tasks (which activity depends on another).	Helps identify the <i>critical path</i> — the sequence of tasks that determines total project time.



Example:

In the online learning project, the activity network might show that testing cannot begin until both backend and frontend development are completed.

3. Risk Management

Risk management is the process of identifying, analyzing, and responding to risks that may affect a project's

schedule, cost, or quality. It ensures that potential problems are recognized early and managed effectively to minimize their impact.

Purpose:

To protect the project from unexpected failures, delays, or cost overruns.

Types of Risks in Software Projects:

a. Project Risks:

- Affect the project schedule or resources.
- *Examples:* Staff leaving mid-project, delay in hardware delivery.

b. Technical Risks:

- Related to implementation, testing, or technical limitations.
- *Examples:* Developers lacking knowledge of a new technology, failure during system integration.

c. Business Risks:

- Affect the organization developing the software.
- *Examples:* New competitors, sudden change in technology, loss of client interest.

Risk Management Process

The process is **iterative**, meaning it continues throughout the project's life. Once plans are made, risks are continuously monitored and updated.

a. Risk Identification:

- List all possible risks that might affect the project.
- Example: "If a developer leaves," "if the server crashes," "if client changes requirements."

b. Risk Analysis:

- Study each risk and estimate its probability and impact.
- Probability levels can be:
 - Very Low (<20%)
 - Low (20–25%)
 - Moderate (26–50%)
 - High (51–75%)
 - Very High (>75%)
- Risks are ranked based on how serious they are.

c. Risk Planning:

- Develop strategies to handle key risks.
- **Avoidance Strategy:** Prevent risk from happening.
Example: Hire backup staff in advance.
- **Minimization Strategy:** Reduce impact if it occurs.
Example: Regularly back up project files.
- **Contingency Plan:** Backup plan if risk happens.
Example: If main server fails, switch to a cloud backup.

d. Risk Monitoring:

- Continuously review identified risks to see if their likelihood or effect has changed.
- Add new risks if new threats appear during development.

4. People Management

People management, also known as **human resource management (HRM)**, is the process of planning, organizing, directing, and controlling human resources in an organization. It ensures that both the organization's goals and employees' needs are met effectively.

Purpose:

To build a motivated, skilled, and efficient workforce that contributes to project success.

Main Functions of People Management:

- **Planning:** Deciding how many people are needed, what skills they should have, and when they are required.
- **Organizing:** Assigning roles and responsibilities to team members.
- **Directing:** Guiding and motivating employees to work efficiently.

- **Controlling:** Evaluating performance and ensuring goals are met.

Example:

In a software development company, HR hires new programmers, assigns them to project teams, provides necessary training, and monitors their productivity.

Key Point:

People management aims to satisfy **organizational goals** (like project success), **individual goals** (like job satisfaction), and **societal needs** (like ethical employment).

5. Proposal Writing

Proposal writing is the process of preparing a **project proposal** to win a contract from a client or customer. It explains what the project aims to achieve, how it will be done, the time required, and the estimated cost.

Purpose:

To convince the customer that the proposed project approach is feasible, efficient, and worth funding.

A Good Project Proposal Includes:

- **Objective of the project** – what the project will achieve.
- **Approach or Methodology** – how the project will be carried out.
- **Cost Estimate** – expected project expenses.
- **Schedule Estimate** – how long each phase will take.

Example:

A software company preparing a proposal for a hospital management system describes the system's features, expected completion time (6 months), required budget, and the development plan to assure the client.

Note:

Proposal writing is a skill that improves with **practice and experience**.

Milestones and Deliverables

- Milestones are **key checkpoints** or **significant progress points** in a project.
- They mark the **end of a specific stage** and help measure progress.
- Often involve short reports or presentations to management.

Example:

Completion of the "System Design" phase or successful completion of "User Interface Prototype" can be milestones.

Deliverables:

- A deliverable is the **final output or result** produced at the end of a major project phase, delivered to the customer.
- It can be a **report, document, software module, or hardware setup**.
- Every deliverable is often tied to a milestone, but not every milestone is a deliverable.

Example:

After completing the "Testing Phase," the deliverable could be a "Test Report" or "Working Application Build."

Difference Between Milestone and Deliverable:

Basis	Milestone	Deliverable
Meaning	A checkpoint showing project progress	A tangible result or output given to the client
Purpose	To track progress	To fulfill project requirements
Example	Completion of the coding phase	Delivery of the complete software system

Software Pricing

Software pricing means deciding how much a customer will pay for the software product or service. It depends not only on cost but also on **market, competition, and business goals**.

Factors Affecting Software Pricing:

1. Market Entry Strategy:

- Companies may quote a low price to enter a new market or attract new clients.

- *Example:* A new startup offers a cheap mobile app to gain visibility.
- 2. Long-Term Profit Expectation:**
 - Some projects may have low immediate profits but can bring bigger opportunities later.
 - *Example:* A company offers a free version of software to build reputation and get future contracts.*
- 3. Uncertain Cost Estimation:**
 - If development costs are unclear, the company might increase the price slightly to cover risks (contingency).
- 4. Unclear Customer Requirements:**
 - When requirements are uncertain or likely to change, companies may lower prices initially to win trust and flexibility.
- 5. Financial Health of Organization:**
 - A company with poor financial stability may reduce prices to secure a project and stay in business.

Estimation Techniques

The **software estimation process** involves predicting how big a project will be, how much effort (time and manpower) it will take, and how much it will cost.

There are two major approaches: **Top-down** and **Bottom-up**.

1. Top-down Approach (Macro Model)

- The total cost is estimated based on overall system characteristics (like total features or complexity), and then this cost is divided among the smaller modules later.
Used when only high-level details are available — usually at the **early stage** of planning.
- **Example:**
Suppose a company is asked to build an **e-commerce website**.
They don't yet know how many modules or pages it will have. So, they estimate the total cost based on similar past projects — say, “a project of this scale usually costs around **Rs. 15,00,000.**”
Later, this amount is divided among modules like “user management,” “product listing,” and “payment gateway.”
- **Advantages:**
 - Covers all system-level functions (like integration and architecture).
 - Quick and simple when little detail is available.
- **Disadvantages:**
 - May overlook hidden complexities (e.g., tricky backend integration).
 - No strong basis to defend the estimate if challenged.

2. Bottom-up Approach

- The cost of each individual module or component is estimated first, and then the total project cost is found by summing them up.
Works best when detailed information about each component is available.
- **Example:**
For the same **e-commerce site**, the team estimates separately:
 - User login module – Rs. 2,00,000
 - Product catalog – Rs. 3,00,000
 - Payment integration – Rs. 4,00,000
 - Admin panel – Rs. 2,00,000
 So, total estimated cost = **Rs. 11,00,000.**
- **Advantages:**
 - More realistic and detailed.
 - Estimation errors in one part can balance out others.
- **Disadvantages:**
 - Takes more time and requires more detailed knowledge.
 - Hard to use early in the project when information is limited.

COCOMO Model (Constructive Cost Model)

The **COCOMO model**, developed by **Barry Boehm**, is a mathematical (algorithmic) model used to **estimate the effort, cost, and time** needed for a software project.

It mainly depends on the **number of lines of code (LOC)** that the software will have.

- 1 KLoC = 1,000 lines of code.

This model helps project managers **predict manpower (person-months)** and **time schedules** early in the project.

Types of Projects in COCOMO

1. Organic Projects

- Simple and small projects.
- The team is small and experienced, and the problem is well understood.
- Example: Payroll systems, billing software, or simple data processing systems.
- **Formula:**
 $\text{Effort} = 2.4 \times (\text{KLoC})^{1.05}$
 $\text{Time} = 2.5 \times (\text{Effort})^{0.38}$

1. Semi-Detached Projects

- Medium-size projects where some team members are experienced while others are not.
- Moderate complexity.
- Example: Database systems, new operating systems, or medium-scale management software.
- **Formula:**
 $\text{Effort} = 3.0 \times (\text{KLoC})^{1.12}$
 $\text{Time} = 2.5 \times (\text{Effort})^{0.35}$

1. Embedded Projects

- Very complex and critical systems requiring high creativity and experience.
- Hardware and software are tightly coupled.
- Example: ATM systems, Air Traffic Control systems, or military-grade control systems.
- **Formula:**
 $\text{Effort} = 3.6 \times (\text{KLoC})^{1.20}$
 $\text{Time} = 2.5 \times (\text{Effort})^{0.32}$

Example Calculation

Given:

- Project size = 200 KLoC
- Team = moderately experienced
→ So, use **Semi-Detached Model**

Step 1: Calculate Effort

$$\begin{aligned}\text{Effort} &= 3.0 \times (200)^{1.12} \\ &= 3.0 \times 378.37 \\ &= \mathbf{1135.12 \text{ Person-Months}}\end{aligned}$$

Step 2: Calculate Development Time

$$\begin{aligned}\text{Tdev} &= 2.5 \times (1135.12)^{0.35} \\ &= 2.5 \times 11.80 \\ &= \mathbf{29.5 \text{ Months}}\end{aligned}$$

Step 3: Average Staff Size

$$\begin{aligned}\text{Average Staff} &= \text{Effort} / \text{Tdev} \\ &= 1135.12 / 29.5 \\ &= \mathbf{38.48 \text{ persons}}\end{aligned}$$

Step 4: Productivity

$$\begin{aligned}\text{Productivity} &= \text{KLoC} / \text{Effort} \\ &= 200 / 1135.12 \\ &= \mathbf{0.176 \text{ KLoC/PM} = 176 \text{ LOC per Person-Month}}\end{aligned}$$

Interpretation:

It means a team of about **38 people** will take **around 29.5 months** to complete the project, writing roughly

176 lines of code per person per month.

Types of COCOMO Models

1. Basic COCOMO

- It is the simplest version.
- Only uses **KLoC (size)** to estimate effort and schedule.
- Accuracy is low because it doesn't consider other factors (like hardware, team skill, etc.).
- **Formula:**
Effort (E) = $a \times (\text{KLoC})^b$
Time (D) = $c \times (E)^d$
Here:
 - E = Effort in Person-Months
 - D = Development time in Months
 - KLoC = Thousands of Lines of Code
 - a, b, c, d = constants (depend on project type)

Example:

For an organic project (10 KLoC), using constants $a=2.4$, $b=1.05$, $c=2.5$, $d=0.38$:

Effort = $2.4 \times (10)^{1.05} = 26.9$ PM

Time = $2.5 \times (26.9)^{0.38} = 9.7$ months

2. Intermediate COCOMO

- More accurate than Basic.
- Considers **extra factors** known as **Cost Drivers**, such as:
 - Product reliability
 - Hardware constraints
 - Team experience
 - Tool usage
- **Formula:**
Effort (E) = $a \times (\text{KLoC})^b \times \text{EAF}$
Time (D) = $c \times (E)^d$
where **EAF = Effort Adjustment Factor**
(Calculated by multiplying ratings of cost drivers. For ideal conditions, EAF = 1.)

Example:

If EAF = 1.2 (slightly higher due to low experience):

Effort = $3.0 \times (200)^{1.12} \times 1.2 = 1362.14$ PM

Time = $2.5 \times (1362.14)^{0.35} = 31$ months

3. Detailed COCOMO

The **Detailed COCOMO model** is the most advanced version.

It includes all the parameters of the **Intermediate model**, but estimates **effort for each phase** of the **Software Development Life Cycle (SDLC)** — such as requirements, design, coding, testing, and maintenance.

It breaks down the total effort phase-wise, so the manager can:

- Track progress more accurately.
- Identify which stage consumes more time and resources.
- Manage risk and cost more effectively.

Formula:

Effort = $a \times (\text{KLoC})^b \times \text{EAF} \times (\text{Phase Weight})$

Each SDLC phase has a **weighting factor** — a percentage of total effort assigned to that phase.

SDLC Phase	Typical % of Total Effort
Requirements & Design	20%
Coding	35%
Integration & Testing	25%

If total effort from Intermediate COCOMO = 500 PM, then:

- Requirements & Design $\rightarrow 0.2 \times 500 = 100$ PM
- Coding $\rightarrow 0.35 \times 500 = 175$ PM
- Integration & Testing $\rightarrow 0.25 \times 500 = 125$ PM
- Maintenance $\rightarrow 0.2 \times 500 = 100$ PM

This helps in **phase-wise resource planning**.

Introduction to Quality Management

Software Quality Management (SQM) ensures that the final software meets the desired level of quality.

It involves continuous monitoring and improvement of processes during software development.

The **goal** is not just to fix defects but to **build a culture of quality** — meaning **every team member** (developers, testers, managers) is responsible for maintaining standards.

Activities of Software Quality Management:

1. Quality Assurance (QA)

- Ensures that the organization follows proper **procedures, methods, and standards** to maintain quality.
- Example: A company may use *ISO 9001* quality standards or *CMMI* levels to guide development practices.

2. Quality Planning

- Decides **which quality standards and procedures** apply to a specific project.
- Example: For a mobile app, the team may decide to follow *Google's UI guidelines* and perform *user testing* every sprint.

3. Quality Control (QC)

- Focuses on **detecting defects** in the actual software product.
- Example: Using automated testing tools like *Selenium* or *JUnit* to check if the software behaves as expected.

Configuration Management (CM)

Configuration Management manages **changes** in the software system — ensuring that every version and update is tracked and consistent.

Without it, developers can lose track of which version is the latest or what changes were made.

Activities in Configuration Management:

1. Change Management

- Handles how change requests (bug fixes, new features) are proposed, approved, and implemented.
- Example: Using *Jira* to track and approve a change in a login module.

2. Version Management

- Maintains records of different versions of code or documents.
- Example: *Git* or *GitHub* are used to manage different versions of source code.

3. System Building

- Combines program components and libraries to build the complete system.
- Example: Running a build pipeline in *Jenkins* to compile and link the code into an executable.

4. Release Management

- Handles packaging and delivery of the software to customers or users.
- Example: Releasing version 2.0 of a mobile app on the Play Store.